Classification Evaluation

CS 485, Spring 2026
Applications of Natural Language Processing

Katrin Erk
College of Information and Computer Sciences
University of Massachusetts Amherst

Evaluation

- Evaluation
  - Test on held-out data
  - What metrics can we use? What makes sense for
    - Unbalanced data?
    - Multiclass?
  - How can we trade off types of errors at runtime, after a model is trained?
- Annotation

False Pos vs False Neg

- Definitions
  Assume two labels: positive and negative
  y: gold standard label
  $\hat{y}$: predicted label
  False positive:
  False negative:

- Are the tradeoffs the same for different applications or tasks?

False Pos vs False Neg

- Definitions
  Assume two labels: positive and negative
  y: gold standard label
  $\hat{y}$: predicted label
  False positive: y=neg, $\hat{y}$ = pos
  False negative: y = pos, $\hat{y}$ = neg

- Are the tradeoffs the same for different applications or tasks?

Evaluation metrics

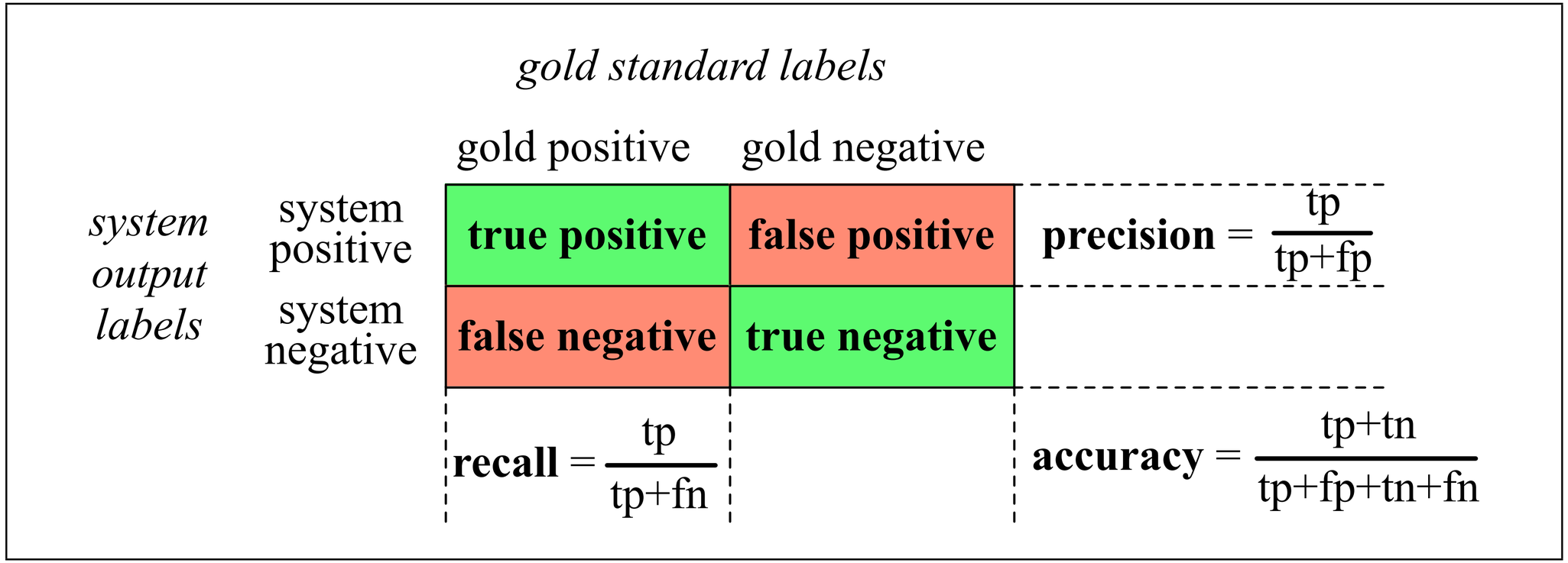


Figure 4.4 A confusion matrix for visualizing how well a binary classification system performs against gold standard labels.

Evaluation metrics

		gold standard labels		
		gold positive	gold negative	
system output labels	system positive	**true positive**	**false positive**	**precision** = $\frac{tp}{tp+fp}$
	system negative	**false negative**	**true negative**	
		recall = $\frac{tp}{tp+fn}$		**accuracy** = $\frac{tp+tn}{tp+fp+tn+fn}$

Figure 4.4 A confusion matrix for visualizing how well a binary classification system performs against gold standard labels.

- Accuracy:
  - But do we care about false positives and negatives equally?
  - What about rare classes?
- Precision, Recall, F1

Decision threshold

- Problem: you'd like a higher precision model (for class SPAM), and willing to sacrifice recall.
- Solution: predict SPAM more conservatively: only if probability exceeds a threshold

With Naive Bayes and logistic regression, can you think of a way to do this?

Decision threshold

- Problem: you'd like a higher precision model (for class SPAM), and willing to sacrifice recall.
- Solution: predict SPAM more conservatively: only if probability exceeds a threshold

Default decision rule: $\hat{y} = 1$ if $P(1 \mid d) > 0.5$

Thresholded decision rule: $\hat{y} = 1$ if $P(1|d) > t$

Visualizing a classifier in feature space

"Bias term"

Feature vector $x = (1,\ \text{count "happy"},\ \text{count "hello"})$

Weights/parameters $\beta = (const,\ large,\ small)$

50% prob where

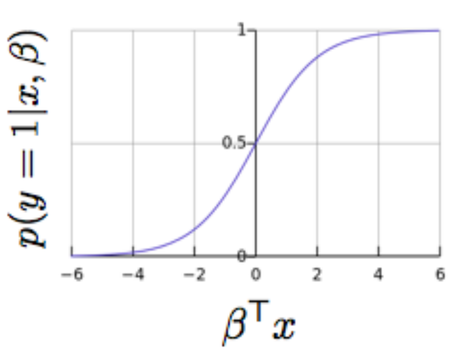
$\beta^\mathsf{T} x = 0$

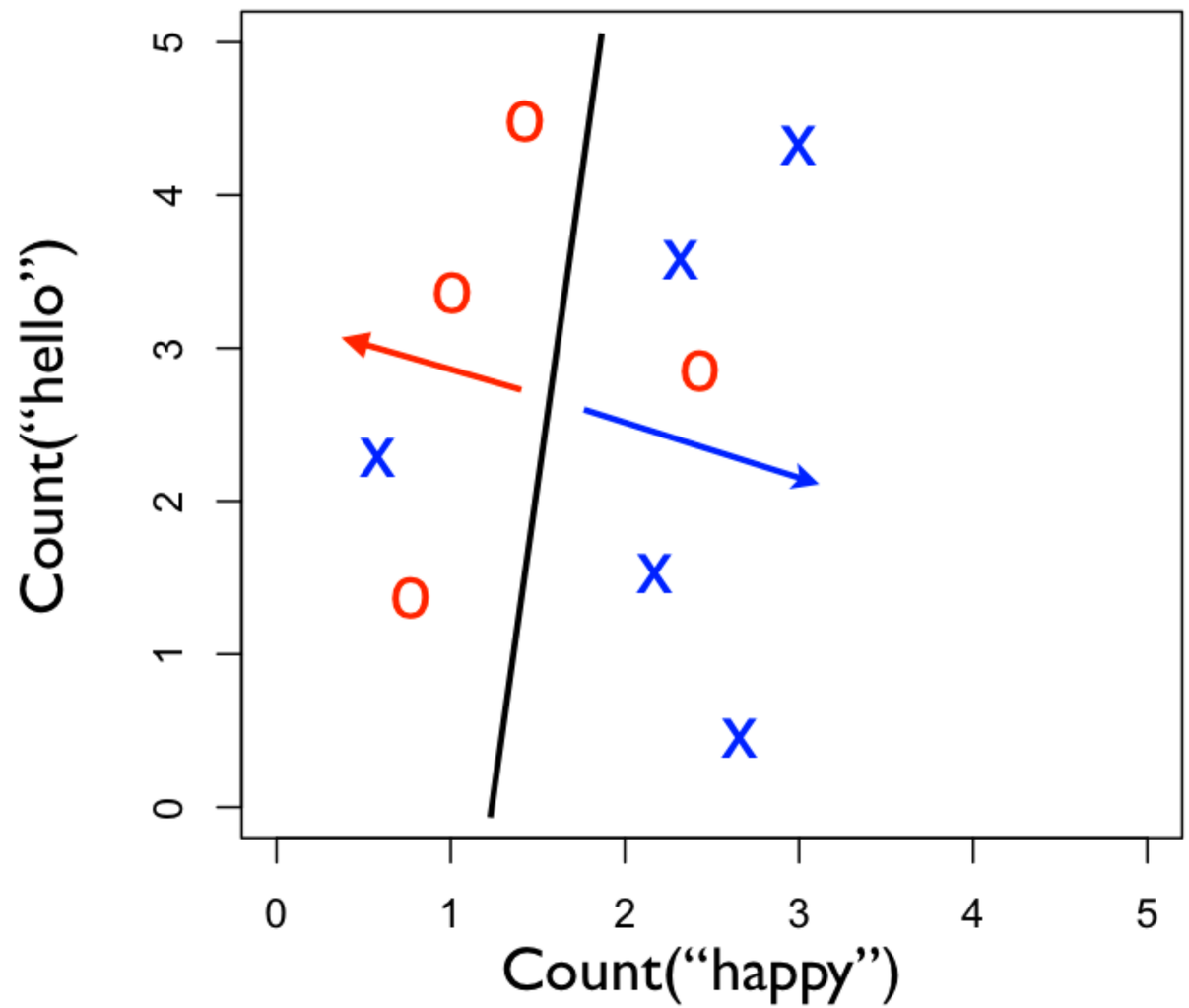
Predict y=1 when

$\beta^\mathsf{T} x > 0$

Predict y=0 when

$\beta^\mathsf{T} x \leq 0$





Decision threshold

- How do errors change as threshold *increases*?

Precision-Recall curve

- Different models may trade off precision and recall
- For a single model, different decision thresholds may trade off precision and recall
- View them jointly with a **precision-recall curve:** precision versus recall plotted for different decision thresholds

Multiclass metrics

- Every class has its own TP, FP, FN counts!

```
from sklearn.metrics import classification_report
>>> y_true = [0, 1, 2, 2, 0]
>>> y_pred = [0, 0, 2, 1, 0]
>>> target_names = ['class 0', 'class 1', 'class 2']
>>> print(classification_report(y_true, y_pred, target_names=target_names))
              precision    recall  f1-score   support

     class 0       0.67      1.00      0.80         2
     class 1       0.00      0.00      0.00         1
     class 2       1.00      0.50      0.67         2

    accuracy                           0.60         5
   macro avg       0.56      0.50      0.49         5
weighted avg       0.67      0.60      0.59         5
```

- Common aggregations: *micro* and *macro* averages. (Tradeoffs?)

Do I have enough labels?

- For training, hundreds to thousands of annotations may be needed for reasonable performance
  - Current work: how to usefully make NLP models with <10 or <100 training examples. "Few-shot learning"
- Exact amounts are difficult to know in advance. Can do a **learning curve** to estimate if more annotations will be useful.

- But where do the labels come from?